



CM3070 Final Project

RL-Powered Stock Advisor

Template: 4.2 Financial Advisor Bot

Student: Max Screawn (ID: 230127041)

Total Word Count: 3580

1. Introduction $\approx$ 610 words

Over the last decade, individuals have changed the way they are investing, low-cost, mobile brokers have made it possible for anyone to buy and sell shares in minutes, with these changes, an increase in the volume of retail trading has been observed. During the same time, a new generation of tools built on machine learning, from "robo-advisors" to large language models, now offer investment suggestions, however frequently with little explanation of *why* a particular action is being recommended.

The increase of tools simplifying the process creates a mismatch as the audience for these is increasing whilst not being experts, self-directed investors, surveys have shown that financial literacy remains low. In the most recent OECD PISA assessment of financial literacy among young people, a substantial portion of 15 year olds performed below the proficiency level of being able to make routine financial decisions, along with this assessment national capability studies find similar gaps within the adult population [19] [20]. For these users, a recommendation is of limited value as they will not understand the reasoning behind it. Unexplained recommendations may be worse than none as it can encourage dependence on a black box methodology rather than building understanding that would let the user properly and critically evaluate the recommendation.

This project is a response to this observed gap, it is a self-hosted desktop application that combines a reinforcement learning (RL) agent with a locally hosted large language model (LLM) to both produce recommendations on buying/holding/selling specific stocks for a specific stock

and then translates the recommendation data into readable text whilst also offering a quiz to test the users understanding. The aim is not to replace a financial advisor or to promise success but to build a low-friction, privacy first educational tool that helps the user understand what he's doing.

This project inherits from the **4.2 Template** (*Financial Advisor Bot*), this project follows the base idea but alters the goal from acting as a simple advisor to an educational tool by not only relying on machine learning but also incorporating an LLM to explore explainable artificial intelligence (XAI), along with these reasons it was also selected as the financial sector is increasingly using more AI systems on a multitude of subjects which makes this topic current and practically relevant for job seeking.

The concrete objectives of the project are:

1. **To build** a desktop application in which a user can train and evaluate a reinforcement learning agent that recommends actions (buy, hold, sell) for a set of stock tickers.
2. **To explain** each recommendation in accessible language, grounded in the technical indicators the agent observed, using a locally hosted LLM.
3. **To educate** the user, through an interactive chat assistant and automatically generated quizzes that test the user's reading of the agent's decisions.
4. **To evaluate** the system from three complementary directions: the performance of the trading model, the quality and groundedness of the explanations, and the experience of real users.

In the following chapters, this report will cover reviews on previous work regarding reinforcement learning on the subject of trading, analysis, large language models and explainable AI, and will compare the project against similar tools (Chp. 2). Following this analysis we will discuss the design of the system including architecture, data models and the file management system strategy (Chp. 3), the next chapter will cover the integration of the design along with the more important algorithms and code (Chp. 4). After the architectural chapters we will move onto the evaluation of the project covering strategy and current results (Chp. 5) before finishing on the conclusion of the current state of the project and considerable paths for improvement in future iterations (Chp . 6).

2. Literature review $\approx$ 1,440 words

This chapter covers the literature that informs the project. It is organised around the three main systems used in this project:

- **reinforcement learning**: for trading, technical and portfolio analysis
- **explainable AI**: using large language models

- **existing tools** that occupy the same space (i.e. robo-advisors)

2.1 Reinforcement learning for trading

The concept of using machine learning to trade by directly interacting with the market is not new, early work formulated trading as a problem of *direct reinforcement* [5], in which the agent learns a policy that maximises a risk adjusted measure of the return rather than learning a predictive model of prices first. This framing is the ancestor of the approach explored within this project.

With the introduction of deep reinforcement learning, the entire field went through drastic acceleration, this covers environment design, reward shaping and evaluation methodology when applied to trading [4]. This identifies several choices that the project inherits directly: using "log returns" as reward signals, comparing the agent against a "buy and hold" benchmark and also evaluating the agent on a different time frame than the one it was tested on. It is important to take into account that financial data is noisy and "non-stationary", meaning that results from one backtest will unlikely generalise to future situations, this point is one of the main considerations in Chapter 5 on the subject of evaluation.

On the algorithmic side, Proximal Policy Optimization (PPO) [1] has become a standard choice for continuous control problems for this task as it's stable, sample-efficient relative to simpler policy gradient methods, and widely supported by mature libraries such as Stable-Baselines3 [2] and the Gymnasium environment interface [3]. PPO's clipped surrogate objective limits how far the policy can change in each update, which is valuable in a domain where overfitting to a short price history is a constant risk.

2.2 Technical and portfolio analysis

In this project, the RL agent acts on *technical indicators*, it favours computing statistics from price and volume history rather than on the raw prices alone, these indicators have a long history in practitioner literature. The relative strength index (RSI) [14] measures whether a stock is overbought or oversold, the MACD histogram [15] uses the difference between two moving averages along with a signal line to emulate momentum in the price change and the Bollinger Bands [16] measure volatility by placing zones around the standard deviation of the averages. None of these indicators is individually predictive, however analysed together they give an interpretable summary of the price behaviour which is what is needed as the observation space for the agent. These details are also important vocabulary that the LLM will use in its explanations.

Portfolio theory provides the foundation for evaluating our model's performance, modern portfolio theory (MPT) formalizes the trade off between risk and return highlighting the importance of diversification [12], whilst metrics like the Sharpe ratio quantify reward per unit of risk [13]. To align with standard financial analysis, this project reports the Sharpe ratio, maximum

drawdown, and returns relative to a buy-and-hold benchmark. Evaluating these parameters ensures the RL agent's performance is judged on a risk-adjusted basis rather than raw return alone.

2.3 Large language models and explainability

The ability to create fluent natural language from structured data makes the explanation layer possible, the transformer architecture [6] supports modern LLMs and the "open weight Llama family [7], [8] is important for this project because it can run locally on consumer hardware using a tool such as Ollama. This local deployment approach is important for two reasons:

- it protects user privacy
- allows the tool to be self hosted without expensive API or hardware costs

The explainability layer connects the RL agent's output to the user. The XAI literature identifies several goals, such as transparency, interpretability, and justification. It also warns that a plausible explanation does not equal a faithful one [9]. Post-hoc, model-agnostic methods like SHAP [10] and interpretable machine learning frameworks [11] form the basis for our design choice: the system does not try to open the neural network itself, but instead shows the inputs (the indicator values) that influenced the decision. The LLM narrates these inputs under strict grounding rules. The risk of the LLM making up numbers is managed by the prompt design outlined in Chapter 4.

Explainability is the bridge between the RL output and the user. The XAI literature distinguishes a number of goals - transparency, interpretability, justification - and warns that a plausible-sounding explanation is not the same as a faithful one [9]. Post-hoc, model-agnostic methods such as SHAP [10] and interpretable machine learning frameworks [11] provide the intellectual background for the design choice made here: the system does *not* attempt to open up the neural network itself, but instead presents the *inputs* (the indicator values) that drove the decision, and has the LLM narrate them under strict grounding rules. The risk that the LLM invents numbers is addressed by the prompt design described in Chapter 4.

A growing line of work applies LLMs directly to finance. BloombergGPT [17] is a large proprietary model trained on financial data, and FinGPT [18] is an open-source alternative intended to make financial LLMs more accessible. Both demonstrate that domain-tuned models can outperform general models on financial tasks, but both also illustrate a tension: they are either very large and expensive to run, or they are positioned as research artefacts rather than as consumer-facing, educational tools.

2.4 Comparable tools and the gap

The justification of this project lies in comparing it with existing tools and comparing their strengths and weaknesses.

Commercial robo-advisors (Betterment, Wealthfront, Nutmeg and similar). These tools are considered as mature and trustworthy in the sense that they automate the construction of the users portfolio by rebalancing investments, optimising costs and taxes, this is done at scale and in a objectively cheap manner. However they are not labeled as being educational tools, all their reasoning is hidden making understanding it a black box scenario, they apparently take a passive risk profile driven approach rather than a per stock recommendation and omit any explanations why specific assets were chose [21]. Their advantage remains convenience whilst their main inconvenience is opacity which is the target of this project.

Academic RL trading agents [4], [5]. These agents are technically strong and progressively more reproducible, however they remain research systems. Their output are actions not explanations, which leads them to not be suited for non-technical users as there are no explanations. Their main strenght is in their methodology on evaluating the market however they do not offer a educational aspect for users which would allow them to understand.

LLM based financial assistants [17], [18]. These assistants provide a genuine solution as an educational tool as they can provide answers to specific questions in detail, however as of the writing of this report these are all majoritarily cloud hosted (a privacy issue), expensive to scale and prone to hallucinations when asked for specific recommendations. They have the strength of a wide understanding of language however this counter acted by the issues they encounter in making decisions grounded on facts regarding the markets evolution.

This project attempts to address the aforementioned gap by providing a solution intersecting the three sectors: an RL agent that makes a real and auditable decision, an LLM that explains that specific decision using the indicator values and a desktop application that attempts to teach the user through chats and quizzes, whilst being offered as a low cost privacy first solution by running locally. This differs from the existing solutions and is the basis for the design of the project.

2.5 Synthesis and the research gap

Three tensions appear repeatedly in the literature, firstly being the persistent trade off between predictive performance and explainability. Models that perform the best on financial benchmarks are usually the least transparent in their reasoning, whilst on the more transparent models which are rule based methods are the least efficient [4] [9]. This project intentionally sacrifices some performance to improve explainability, this is justifiable as the main goal of this project is education not profit.

Along with these tensions, the move towards open weight locally hostable modes [7], [8] has changed the economics of explainability. Now a reliable, privacy preserving explanation layer does not need an expensive cloud API. Furthermore, evaluation practices vary significantly between communities, RL trading work is assessed quantitatively through backtests, while XAI

and human-computer interaction fields require faithfulness checks and user studies. A system that bridges both fields, like this one, must meet the standards of both types of evaluations, this need drives the combined quantitative and user study approach detailed in Chapter 5.

The gap mentioned in Chapter 2.4, no existing tool integrates an auditable decision, grounded explanation and educational interface in a self hosted desktop application, feeds into the three listed tensions.

3. Design $\approx$ 820 words

The objectives are turned into a specific design in this chapter, which starts by outlining the requirements, then goes on to describe the architecture, the data storage, and the end-to-end data flow.

3.1 Requirements

From the analysis of users in Chapter 1, the following functional requirements were derived:

- **FR1.** The user has the option of choosing a ticker and a date range, after which a new RL agent can be trained.
- **FR2.** The models that have been trained can be seen in the interface, selected and deleted.
- **FR3.** It is possible to assess a chosen model during a separate time period in order to generate individual recommendations for each ticker and to produce a backtest summary.
- **FR4.** The recommendations are given in simple language and refer to the technical indicators that were observed.
- **FR5.** A user is able to have a chat with an educational assistant, and the conversation history is retained between sessions.
- **FR6.** The system is capable of creating a quiz based on the most recent recommendations and then checking the user's answers.

The non-functional requirements are: **self-hosting** (since all machine learning and language processing is carried out locally), **low setup friction** (the user should not have to start up separate services), **privacy** (no data leaves the machine), and **safety** (the tool must appear to be educational rather than giving financial advice).

3.2 Architecture

The architecture is layered, with each layer responsible for a single concern. Figure 1 shows the overall structure.

with there being many relevant librairies (such as: Stable-Baselines3, Gymnasium, Backtrader) for these tasks. This keeps the desktop layer simple and places the complexity where it fits best, but this means having to manage another runtime, we further discuss this trade off in Chapter 5.

3.3 Data storage

The feedback on the preliminary report asked, in particular, *what is in the data storage* and *where the historical data is*, this sub-chapter will provide answers for these.

There are **two** storage mechanisms, serving different purposes.

SQLite (relational, persistent). A SQLite database is used for providing a structured system that allows to persist data storage across sessions. Table 1 lists the schema, which is created by versioned migrations in the Rust backend.

Table	Columns	Purpose
<code>channels</code>	<code>id</code> , <code>name</code> , <code>created_at</code>	Named chat channels (e.g. "General")
<code>conversations</code>	<code>id</code> , <code>channel_id</code> , <code>title</code> , <code>created_at</code>	Conversations within a channel
<code>messages</code>	<code>id</code> , <code>conversation_id</code> , <code>role</code> , <code>content</code> , <code>created_at</code>	Individual chat messages (user/assistant/system)
<code>trained_models</code>	<code>id</code> , <code>model_name</code> , <code>tickers</code> , <code>start_date</code> , <code>end_date</code> , <code>created_at</code>	Registry of models trained via the UI

*Table 1 - SQLite schema. This is the persistent store for chat history and the model registry; it does **not** hold market data or model weights.*

Filesystem (application data directory). Since the machine-learning artefacts are both too large and too diverse for a relational database, they are stored on the disk in the directory that the operating system provides for application data and which is then passed to Python using a `--data-dir` argument. In this directory, the `output/` folder contains the trained models (`ppo_trading_model_<timestamp>.zip`) and the evaluation payload (`recommendations.json`). This arrangement directly tackles the weakness identified in the midterm: the original prototype had relied on the directory that was currently set, while the new design makes use of one clear root directory together with distinctly named subfolders for the models and the outputs.

Historical market data. The historical prices are not stored at all; instead, they are downloaded from Yahoo Finance using `yfinance` at the beginning of each training and evaluation session and are kept in memory as pandas DataFrames. This approach results in low storage requirements and up-to-date data, but it means the system relies on network connectivity and

on the provider's API. An improvement that will be introduced in Chapter 6 is to cache the downloaded series in the `data/` directory so that the experiments can be carried out offline.

The system ensures path safety through the use of a small `StorageManager` which resolves each relative path in relation to the data directory and discards any path that is resolved to lie outside of it, thus avoiding any accidental writing outside the application's own folder.

3.4 Data flow and API

Figure 2 shows the end-to-end workflow as a sequence of stages, each triggered through an explicit API endpoint.

```
[train]  download market data -> build indicators -> PPO training -> save model .zip
[evaluate] load model -> download eval data -> Backtrader backtest -> allocations
        -> recommendations.json (metrics + allocations + evolution curves)
[chat]   user question -> Ollama -> reply (history persisted in SQLite)
[quiz/generate] recommendations.json -> deterministic questions (answers stripped)
[quiz/review]   questions + user answers -> Ollama -> score + feedback (JSON)
```

Figure 2 - End-to-end pipeline. Stages communicate through the filesystem hand-off (`model .zip` then `recommendations.json`), which is why path consistency is critical. Table 2 lists the endpoints that drive these stages.

Endpoint	Method	Input	Output
<code>/train</code>	POST	tickers, date range, model name	status, valid tickers
<code>/evaluate</code>	POST	model name, tickers, dates, budget	backtest metrics + allocations
<code>/chat</code>	POST	message (+ optional history)	assistant reply
<code>/quiz/generate</code>	GET	-	questions with correct answers removed
<code>/quiz/review</code>	POST	user's answers	score and per-question feedback
<code>/health</code>	GET	-	liveness check

Table 2 - FastAPI endpoints exposed by the sidecar. Inputs are validated with Pydantic models; the two quiz endpoints are designed so the correct answers never reach the client.

The design intentionally keeps the stages separable: each endpoint can be exercised independently, which makes testing possible at each boundary and keeps failures localised.

4. Implementation $\approx$ 980 words

In this chapter, we will be covering the implementation of the projects design, following the structure from the Python sidecar (Figure) outwards, focusing on the more important algorithms and code.

4.1 Market data and technical indicators

Data acquisition is implemented in `rl_pipeline/data.py`. For each ticker, `yfinance` downloads OHLCV bars over the requested range; series with too few rows are discarded. The function `add_indicators` then appends four features: RSI-14 [14], a MACD histogram (12/26/9) [15], Bollinger Band position (20-day, 2σ) [16], and ATR-14. These five columns, the close price and four indicators, form the feature vector `FEATURE_COLS` used everywhere downstream, so the same signals the agent learns from, are the ones the LLM later uses to generate the explanations.

4.2 The trading environment

The environment in `rl_pipeline/environment.py` is a custom `gymnasium.Env`. Its observation is done for each ticker, by using a rolling window of the last `LOOKBACK` (20) timesteps of the feature vector, then normalising the data per-window to $[0, 1]$ and concatenating it with the current portfolio weights and the cash fraction. The action space is a continuous vector in $[-1, 1]$ per ticker: values above $+0.05$ buy (capped at 40% of portfolio value per position), values below -0.05 sell, and the rest hold. A 0.1% commission is applied to each trade to emulate possible costs for trading platforms.

The reward is the central design decision. Each step returns the log return of the portfolio:

```
reward = math.log(new_v / (self._prev_v + 1e-9))
```

Using log returns (rather than raw profit) makes the reward proportional and symmetric, whilst discouraging the agent from being satisfied with small absolute gains, which matches the common practice in the RL trading literature [4], [5].

4.3 Training with PPO

Training uses Stable-Baselines3's PPO with an MLP policy (`MlpPolicy`) and a two-layer network of 128 and 64 units. Table 3 lists the hyperparameters, in regards to the total timestep budget

being a low 30,000, this was decided to maintain rapid which is deliberately small so that training completes in minutes on a laptop rather than hours.

Hyperparameter	Value
Learning rate	3×10^{-4}
Rollout steps (<code>n_steps</code>)	256
Batch size	64
Epochs	10
Discount (γ)	0.99
Entropy coefficient	0.01
Network	[128, 64]
Total timesteps	30,000

Table 3 - PPO hyperparameters. The small timestep budget trades peak performance for usability: a non-technical user can train a model in minutes.

The trained model is serialised to the data directory and a record is inserted into the `trained_models` table (Table 1) so the UI can list it.

4.4 Backtesting and recommendation generation

The saved model is loaded and then executed over a separate time period using Backtrader. The RLStrategy reproduces the same observation vector at every bar and carries out the agent's action, whereas the BuyAndHold strategy divides its investment equally among the different tickers on the first bar just as the benchmark does. The `run_backtest_suite` function adds analysers for the Sharpe ratio, drawdown, trades and time-return and retrieves the summary metrics, namely the RL return, the buy-and-hold return, alpha (defined as the difference), the Sharpe ratio, the maximum drawdown, the annualised volatility and the number of trades.

The last recommendation is generated by `_calculate_allocations`, which requests an action from the model at the final timestamp and then transforms it into per-ticker allocations (specifying whether it is BUY/SELL/HOLD, the dollar amount, the percentage of the budget, and the observed RSI/MACD/Bollinger values). The entire payload – comprising the metrics, the allocations and the evolution curves – is saved to `recommendations.json`:

```

payload = {
    "budget": budget,
    "cash_remaining": round(budget - total_out, 2),
    "eval_period": f"{eval_start} -> {eval_end}",
    "backtest_summary": { ... },
    "allocations": [a.__dict__ for a in allocations],
    "evolution": evolution,
}

```

`build_evolution_curves` converts the daily time returns into aligned equity and drawdown series that the frontend can plot directly.

4.5 The explanation layer

ToDo: reformulate The explanation layer is built around an `OllamaClient` wrapper that talks to a locally bundled Ollama server, having a given structure to the wrapper will allow us to add in compatibility for other external providers if the user wishes to use one (i.e. Claude, ChatGPT, etc...). For this project, I found that using the `0xroyce/plutus` model (a Llama-3.1-based model [8]) was well suited in providing financial explanations and reasoning. Regarding the prompting of the LLM, three distinct system prompts were tailored to properly guide the reasoning of the model:

- **RECOMMENDATIONS:** instructs the model to explain a recommendation using *only* the supplied indicator data, to mention risks and counterpoints, and to refuse to fill gaps with invented numbers.
- **CHAT:** an educational tutor that teaches concepts, defines terms, and refuses to give personalised financial advice.
- **QUIZ_REVIEW:** instructs the model to return a strict JSON object scoring each answer.

For each allocation in `recommendations.json`, `generate_explanation` builds a prompt containing the ticker, the action and the indicator values, asks the LLM to explain it in plain language, and stores the result as a structured `Explanation`. This is the direct link between the RL output and the user-facing text.

4.6 The quiz

ToDo: quiz system expansion, currently only a basic implementation with predefined questions.

The quiz system currently turns the latest recommendations in a short 3 question quiz: a recall question per top allocation ("which action did the agent recommend for this ticker?"), a benchmark question ("did the agent outperform buy-and-hold?"), and an open question asking the user to explain the highest-weighted allocation. The correct answers are stripped before the quiz is sent to the client and regenerated server-side at review time, so the client cannot simply

read the answers. Once the answers are sent back to the server, these will be reviewed by the LLM using the QUIZ_REVIEW schema, which returns a score and per-question feedback.

4.7 The desktop shell

The Rust backend (`src-tauri/src/main.rs`) registers the SQLite migrations and, in production, spawns the Python sidecar with the data directory as an argument:

```
let sidecar = app.shell().sidecar("app").unwrap()
    .arg("--data-dir").arg(data_dir_str);
let (mut rx, _child) = sidecar.spawn().expect("failed to spawn python sidecar");
```

The output is sent to the logs and the sidecar's CORS policy is limited to the Tauri origins. The frontend, which uses React and Vite together with shadcn-style components based on Base UI primitives and Recharts for charting, includes five views: Chat, Evaluate, Predictions, Settings, and a Dashboard placeholder. The Predictions page (Figure 3) displays the equity and drawdown curves as well as the allocation breakdown derived from the JSON evolution data and also contains the quiz. The Evaluate page (Figure 4) shows the backtest metric cards and the allocations table, these being the main means by which a user can examine model performance. Chat and training are illustrated in Figure 5: the chat retains its history using the SQLite tables listed in Table 1, and the training form submits data to '/train' (as specified in Table 2).

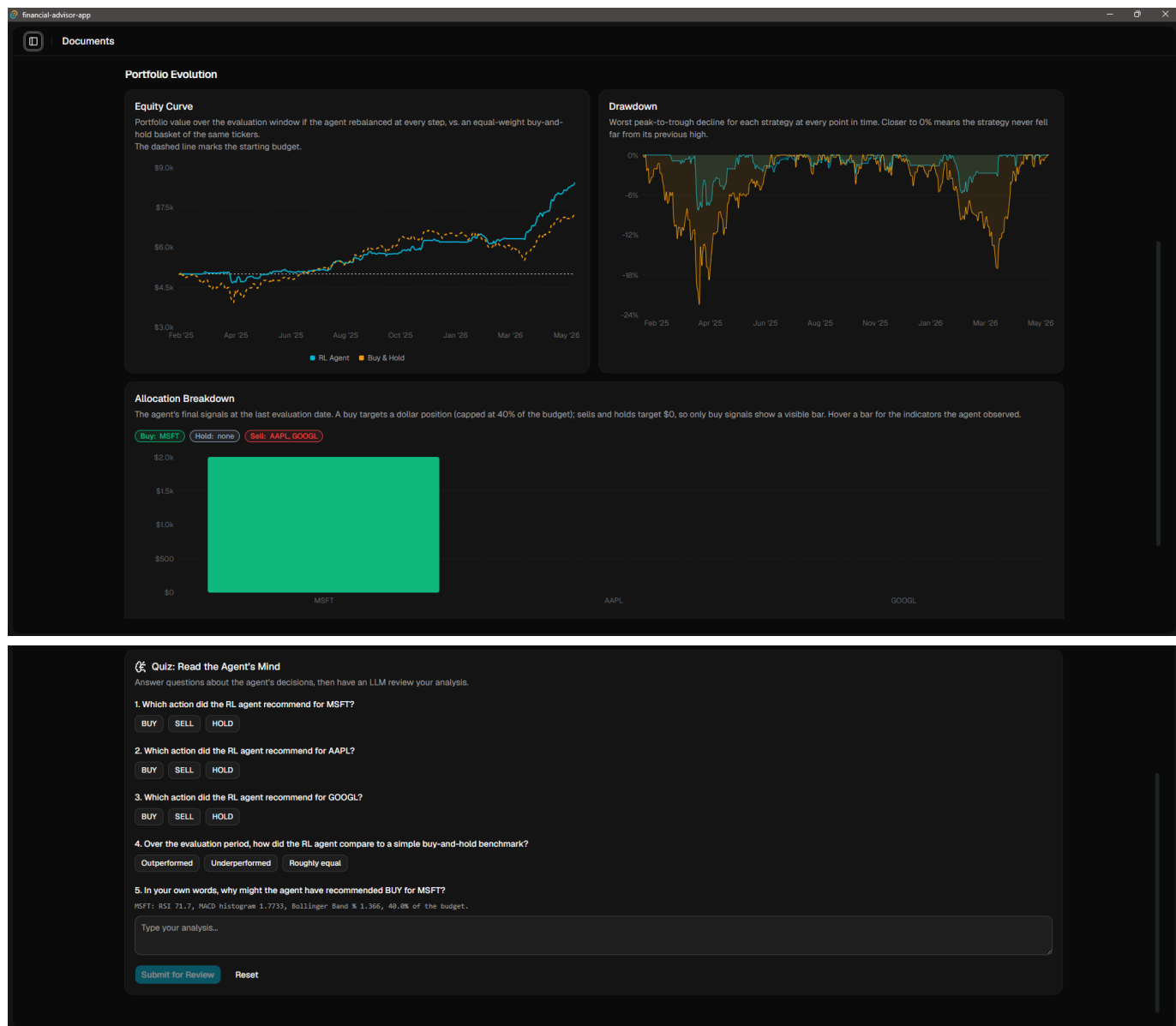


Figure 3 - The Predictions page. It plots the agent's equity curve against the buy-and-hold benchmark, the drawdown series, and the dollar allocation per ticker, and hosts the quiz below.

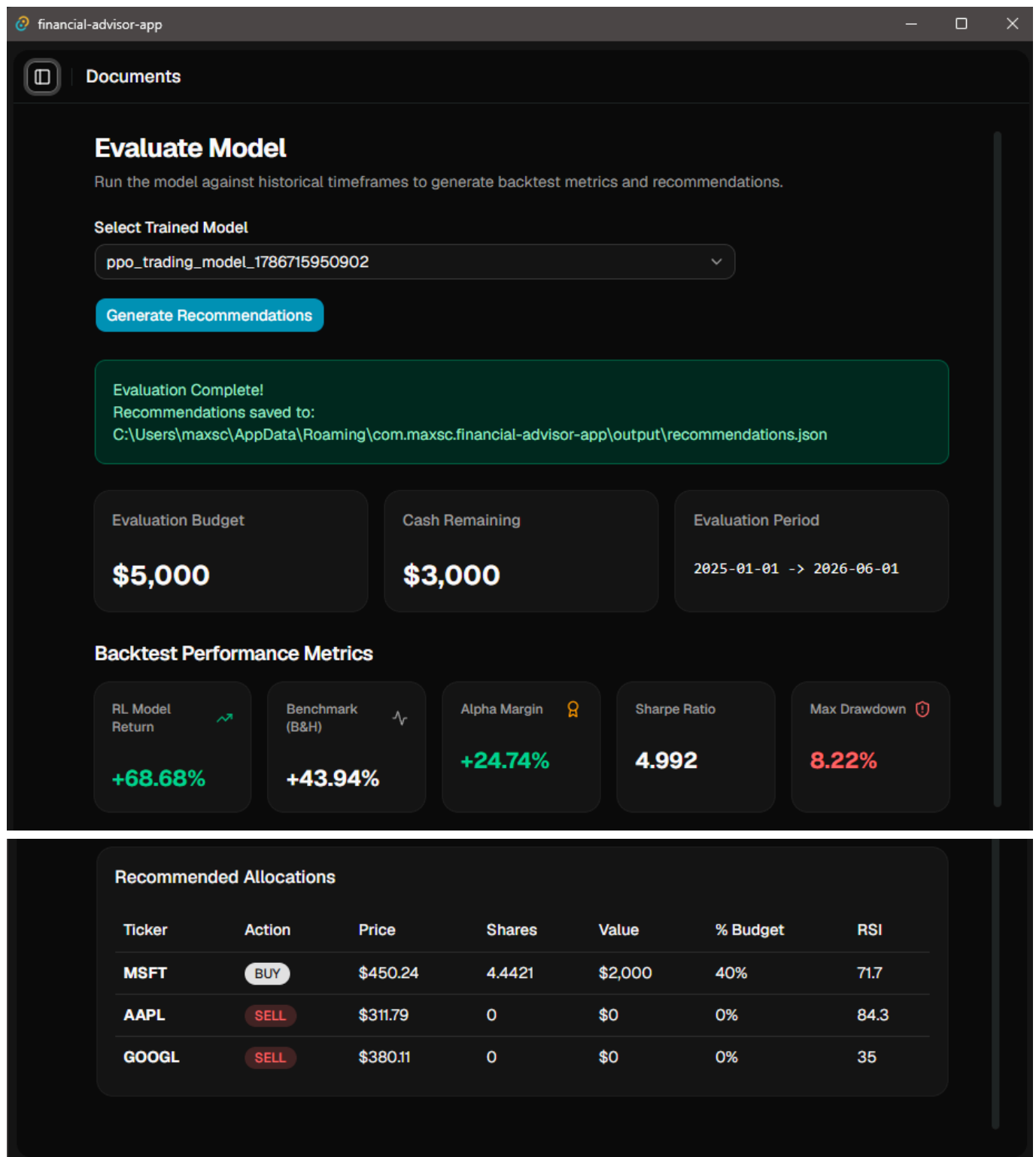


Figure 4 - The Evaluate page. The five metric cards (RL return, benchmark, alpha, Sharpe, drawdown) and the allocations table are the primary way a user inspects model performance.

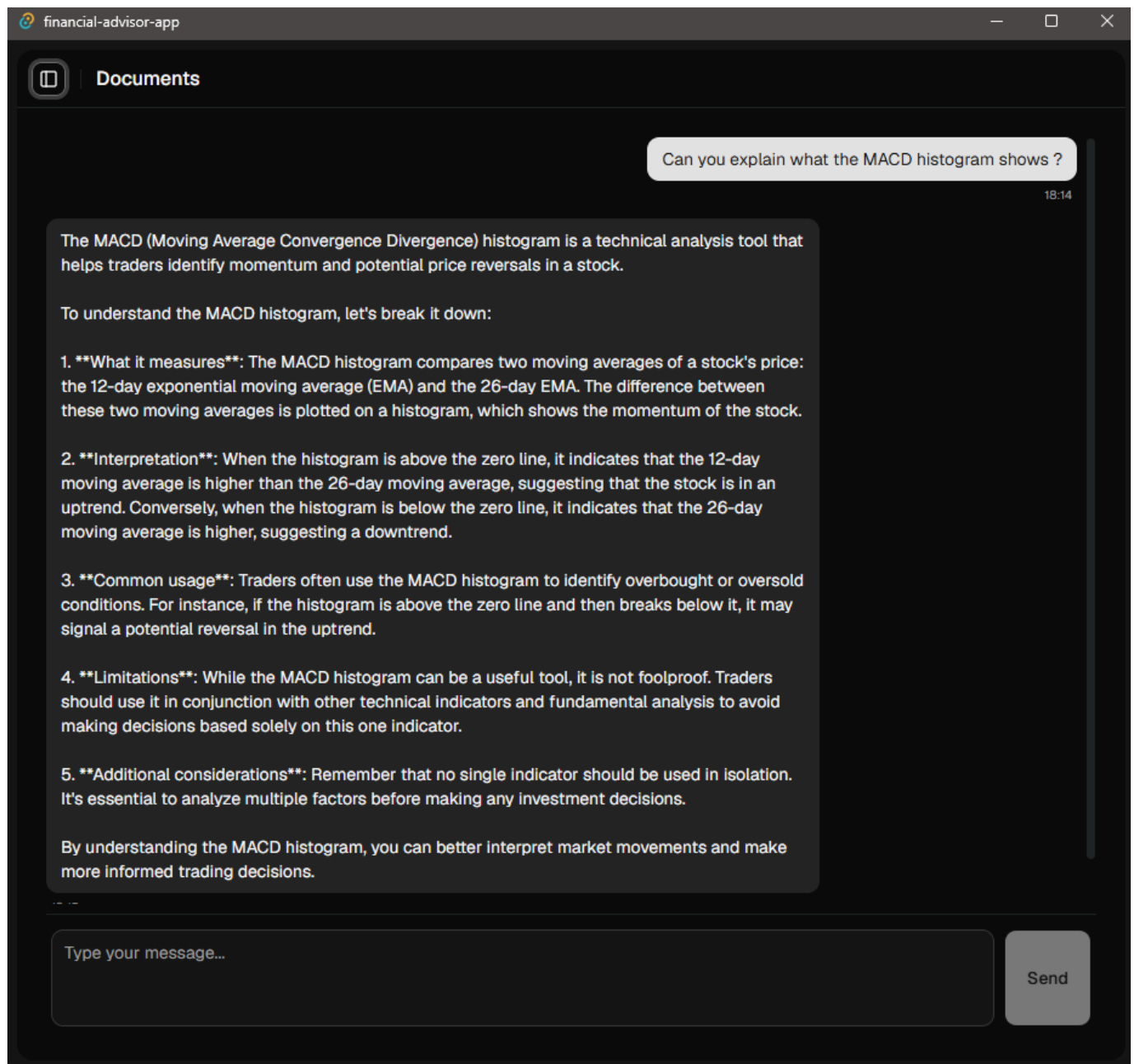


Figure 5 - The Chat and training interfaces. Chat persists history through the SQLite tables of Table 1; the training form feeds `/train` (Table 2).

5. Evaluation $\approx$ 810 words

ToDo: evaluation steps need to be better considered, currently too much was evaluated in a "production" aspect but without sufficiently tracking metrics and possible user interaction issues. The evaluation was deemed insufficient in the preliminary report, a new read through is required on the feedback to address the issue and properly work on this section prior to continuing development

5.1 Evaluation strategy

The project has three distinct aims, a *working* model, *faithful* explanations, and an *educational* experience, and each requires its own kind of evidence. Table 4 maps each aim to a method, the metric it produces, and the point in the project at which it is applied.

Aim	Method	Metric	Status
Model quality	Backtesting on held-out data	Return, alpha vs buy-and-hold, Sharpe, max drawdown	TBD
Model quality	Technical/unit testing of pipeline	Passing test suite; edge-case coverage	TBD
Explanation quality	Groundedness checks vs recommendations.json	% of explanations free of invented figures	TBD
Educational value	User study: task + questionnaire	SUS score [26], task completion rate, quiz scores	TBD

Table 4 - Evaluation strategy. Each aim is evaluated by a method appropriate to it, the model is evaluated quantitatively, while the user experience is evaluated with human participants.

5.2 Model performance evaluation

The primary quantitative evaluation is the backtest described in Chapter 4: the agent is trained on one period (2021–2023 by default) and evaluated on a disjoint, later period (2025–2026 by default), and compared against an equal-weight buy-and-hold benchmark. The metrics reported are total return, alpha (return above the benchmark), Sharpe ratio (annualised, risk-free rate 5%/252 per day) and maximum drawdown.

Table 5 shows the intended presentation format, with representative placeholder values that must be replaced with actual results from a stable run before submission.

Metric	RL agent	Buy-and-hold
Total return (%)	[+ to fill]	[+ to fill]
Alpha (%)	[+ to fill]	-
Sharpe ratio	[+ to fill]	[+ to fill]
Max drawdown (%)	[+ to fill]	[+ to fill]
Trades closed	[+ to fill]	0 (passive)

Table 5 - Backtest results (example layout). The two-column format makes the agent's risk-adjusted performance directly comparable with the benchmark, which is the point the reader should look for.

Two methodological precautions follow from the literature [4]. First, training and evaluation periods never overlap, so the results are not contaminated by memorisation. Second, because both PPO training and market conditions are stochastic, the final report will report results over several seeds and several evaluation windows rather than a single "lucky run".

5.3 Technical testing

ToDo: unit tests need to be considered and implemented

Technical testing verifies that the system is *correct*, independent of whether the strategy is profitable:

- **Pipeline unit tests (pytest).** Tests for `add_indicators` (expected values on synthetic data), the environment (observation shape, action-to-position mapping, commission accounting), and `build_evolution_curves` (alignment of the two series).
- **Quiz determinism.** A test that the same `recommendations.json` always produces the same questions and correct answers, and that `strip_correct_answers` removes the answer field.
- **API integration tests.** Tests that `/train`, `/evaluate`, `/quiz/generate` and `/quiz/review` respond with the documented Pydantic schemas and return 4xx/5xx errors rather than crashing on bad input (e.g. a missing model name).
- **Error-path tests.** Verifying the file-handling behaviour identified as the midterm's key weakness: evaluation against a non-existent model returns a clear 404, and the `StorageManager` rejects paths that escape the data directory.

At the time of writing these tests are specified and not yet implemented, completing them is a priority of the remaining work (Chapter 6), because the midterm feedback identified robustness and testing as the main gaps.

5.4 Explanation quality

A model that is correct is not sufficient; its explanations have to be grounded. Since the prompt tells the LLM to use only the data provided, it is possible to automatically check groundedness: for each explanation, extract any figures it mentions and then verify that these figures appear in the relevant entry in `recommendations.json`. The aim is to find the proportion of explanations that do not introduce any made-up figures. This is an instance of the post-hoc faithfulness checks referred to in the XAI literature [9], [10]. Nevertheless, human judgement is required regarding tone and teaching method, as the user study addresses this.

5.5 User evaluation

ToDo: engage in proper user study and testing, point was highlighted in the preliminary report feedback. The used development approach didn't properly consider it, as it was developing an MVP without user feedback.

5.6 Critical assessment so far

ToDo: write section, points to cover:

- Core systems are achieved
- Limitations that haven't been addressed
- Observed/Noticed security risks, inconsistencies

6. Conclusion ≈N/A words

ToDo: write section, points to cover:

- Recap the built project
- Cover achieved capabilities & limitations
- Future plans, i.e. improve testing, improve accuracy and feedback

References

ToDo: some these were used in prior versions of this report but I didn't properly clean up or possibly missed mentions

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[2] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable Reinforcement Learning Implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.

[3] M. Towers et al., "Gymnasium: A Standard Interface for Reinforcement Learning Environments," *arXiv preprint arXiv:2407.17032*, 2024.

[4] Z. Zhang, S. Zohren, and S. Roberts, "Deep Reinforcement Learning for Trading," *The Journal of Financial Data Science*, vol. 2, no. 2, pp. 25–40, 2020.

[5] J. Moody and M. Saffell, "Learning to Trade via Direct Reinforcement," *IEEE Transactions on Neural Networks*, vol. 12, no. 4, pp. 875–889, 2001.

- [6] A. Vaswani et al., "Attention Is All You Need," in *Advances in Neural Information Processing Systems*, 2017.
- [7] H. Touvron et al., "Llama 2: Open Foundation and Fine-Tuned Chat Models," *arXiv preprint arXiv:2307.09288*, 2023.
- [8] A. Grattafiori et al., "The Llama 3 Herd of Models," *arXiv preprint arXiv:2407.21783*, 2024.
- [9] A. Adadi and M. Berrada, "Peeking Inside the Black-Box: A Survey on Explainable Artificial Intelligence (XAI)," *IEEE Access*, vol. 6, pp. 52138–52160, 2018.
- [10] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems*, 2017.
- [11] C. Molnar, *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*, 2nd ed., 2022. [Online]. Available: <https://christophm.github.io/interpretable-ml-book/>
- [12] H. Markowitz, "Portfolio Selection," *The Journal of Finance*, vol. 7, no. 1, pp. 77–91, 1952.
- [13] W. F. Sharpe, "The Sharpe Ratio," *The Journal of Portfolio Management*, vol. 21, no. 1, pp. 49–58, 1994.
- [14] J. W. Wilder, *New Concepts in Technical Trading Systems*. Trend Research, 1978.
- [15] G. Appel, *Technical Analysis: Power Tools for Active Investors*. FT Press, 2005.
- [16] J. Bollinger, *Bollinger on Bollinger Bands*. McGraw-Hill, 2001.
- [17] S. Wu et al., "BloombergGPT: A Large Language Model for Finance," *arXiv preprint arXiv:2303.17564*, 2023.
- [18] H. Yang, X.-Y. Liu, and C. D. Wang, "FinGPT: Open-Source Financial Large Language Models," *arXiv preprint arXiv:2306.06031*, 2023.
- [19] OECD, *PISA 2022 Results (Volume IV): Students' Financial Literacy*. OECD Publishing, 2024.
- [20] FINRA Investor Education Foundation, *The State of U.S. Financial Capability: The 2021 National Financial Capability Study*, 2022.
- [21] D. Jung, V. Dorner, C. Weinhardt, and H. Puzmaz, "Designing a Robo-Advisor for Risk-Averse, Low-Budget Consumers," *Electronic Markets*, vol. 28, pp. 367–380, 2018.
- [22] Ollama, "Ollama documentation." [Online]. Available: <https://docs.ollama.com>. Accessed: Aug. 15, 2026.
- [23] Backtrader Community, "Backtrader documentation." [Online]. Available: <https://www.backtrader.com>. Accessed: Aug. 15, 2026.

[24] yfinance developers, "yfinance: Yahoo! Finance market data downloader." [Online]. Available: <https://github.com/ranaroussi/yfinance>. Accessed: Aug. 15, 2026.

[25] Tauri Contributors, "Tauri 2 documentation." [Online]. Available: <https://tauri.app>. Accessed: Aug. 15, 2026.

[26] J. Brooke, "SUS: A 'Quick and Dirty' Usability Scale," in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, Eds. London, U.K.: Taylor & Francis, 1996, pp. 189–194.